

Motor de Ignición Financiera — Cómo llega la información y cómo se muestra

Motor de Ignición Financiera — Cómo llega la información y cómo se muestra

1. Panorama general
2. Cómo llega la información (el backend)
3. Cómo se muestra la información (el frontend)
4. Resumen: quién ve qué

Motor de Ignición Financiera — Cómo llega la información y cómo se muestra

Para: Equipo de desarrollo **Propósito:** Documentar el recorrido completo de un dato, desde que existe como una transacción cruda en Nessie hasta que aparece en el panel del usuario — y las distintas formas en que esa misma información se puede presentar.

1. Panorama general

Todo el sistema tiene un solo flujo de información con cuatro pasos, y ese mismo resultado final se puede mostrar de cuatro maneras distintas según quién lo esté viendo.

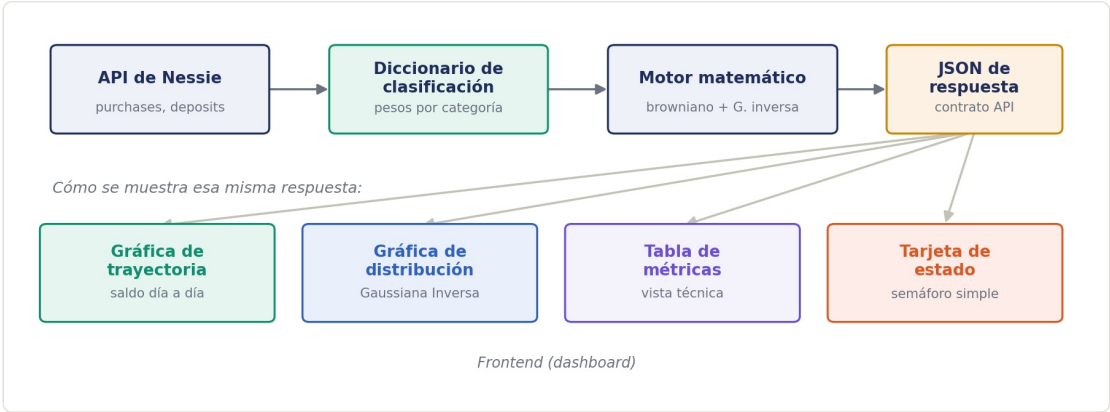


Diagrama de flujo de datos

Los primeros cuatro recuadros son el **backend**: de ahí sale un único JSON. Los siguientes cuatro recuadros son el **frontend**: distintas vistas que consumen ese mismo JSON, cada una pensada para una audiencia distinta (el jurado técnico, el desarrollador depurando, o el dueño de la PyME que solo quiere saber si está en problemas).

2. Cómo llega la información (el backend)

Paso 1 — API de Nessie

El backend hace `GET` a los endpoints de `purchases`, `deposits` y `merchants` de la cuenta. Esto regresa una lista de transacciones crudas con fecha, monto y el `merchant_id` de cada compra — sin ninguna clasificación de riesgo todavía.

Paso 2 — Diccionario de clasificación

Cada transacción se cruza con el diccionario de pesos que el equipo definió (`CATEGORY_WEIGHTS`). Este paso convierte “compras a distintos comercios” en un solo número por día: el flujo neto ponderado. Es la única parte del pipeline donde importa *qué* se compró — después de este paso, esa información se descarta.

Paso 3 — Motor matemático

Con la serie de flujo diario ya ponderada, se calculan $\mathbf{B}$, μ y σ , y se alimentan a la fórmula de la Gaussiana Inversa (documentada a fondo en el documento técnico del modelo). El resultado es una distribución de probabilidad completa sobre el día en que el saldo cruzaría cero.

Paso 4 — JSON de respuesta

Este es el contrato real entre el backend y cualquier vista que lo consuma. Así se ve exactamente:

```
{
  "account_id": "5a1b0e4e4f523604900000002",
  "fecha_calculo": "2026-01-15",
  "dia_observado": 15,
  "parametros": {
    "B0": 5000.00,
    "mu": -266.52,
    "sigma": 237.30,
    "ventana_dias": 7
  },
  "prediccion": {
    "dia_esperado_quiebra": 18.76,
    "intervalo_confianza_90": [13.15, 25.69],
    "intervalo_confianza_50": [16.01, 21.09]
  },
  "riesgo": {
    "fecha_finiquito": "2026-01-18",
    "prob_quiebra_antes_finiquito": 0.46,
    "umbral_alerta": 0.35,
    "estado": "riesgo_alto",
    "webhook_disparado": true
  },
  "serie_saldo": [
    {"dia": 0, "saldo": 5000.00},
    {"dia": 1, "saldo": 4508.36},
    {"dia": 15, "saldo": 1002.20}
  ]
}
```

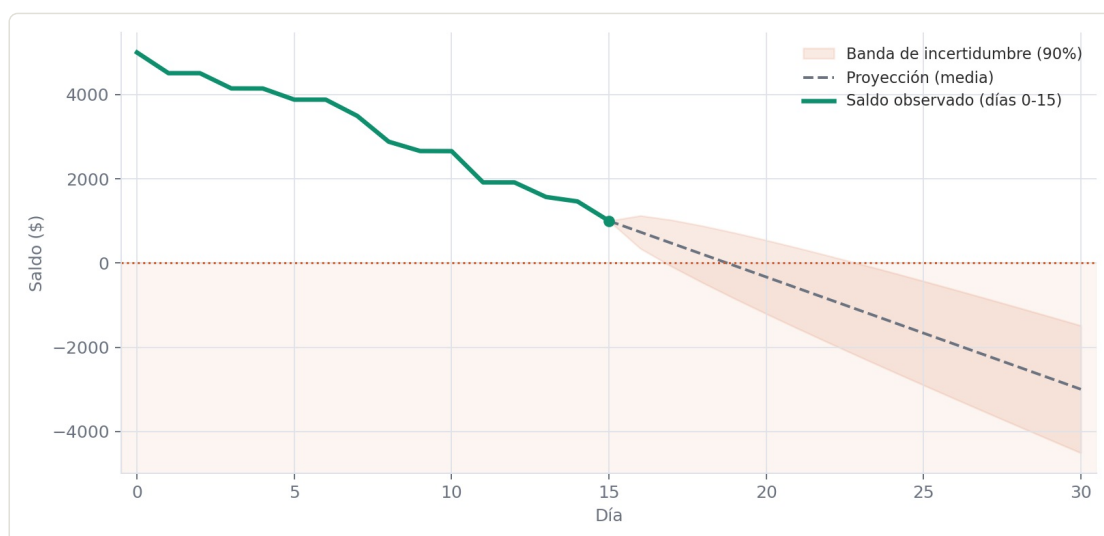
Punto importante para el equipo: ninguna de las cuatro vistas que siguen recalcula nada. Todas leen este mismo JSON y solo deciden cómo dibujarlo. Si el número está mal, está mal en las cuatro vistas por igual — lo cual es bueno, porque significa que solo hay un lugar donde puede haber un bug de cálculo.

3. Cómo se muestra la información (el frontend)

3.1 Gráfica de trayectoria

Consume `serie_saldo` del JSON. Dibuja el saldo observado día a día como una línea sólida, y extiende una **proyección punteada** más allá del último día observado usando `parametros.mu` y `parametros.sigma`

— esa banda sombreada alrededor de la proyección es la incertidumbre del modelo haciéndose visible: entre más lejos se proyecta, más ancha se pone, porque el ruido browniano se acumula con el tiempo.

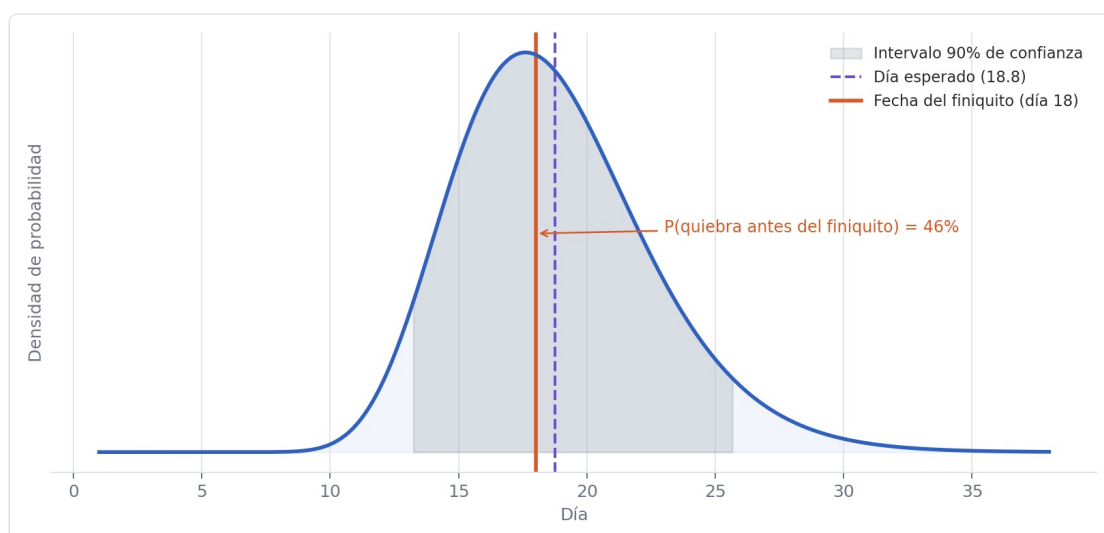


Gráfica de trayectoria del saldo

La franja roja tenue debajo de cero marca la “zona de quiebra”. En el dashboard interactivo, esta gráfica se anima día por día (como el time-lapse del pitch) usando un botón de reproducción que va revelando `serie_saldo` un punto a la vez.

3.2 Gráfica de distribución (Gaussiana Inversa)

Consume `prediccion` y `riesgo` del JSON. No muestra datos observados — muestra la distribución de probabilidad completa calculada por el modelo. El área sombreada gris es el intervalo de confianza del 90% (`intervalo_confianza_90`); la línea vertical morada marca el día esperado; la línea naranja marca `fecha_finiquito` .



Gráfica de la distribución Gaussiana Inversa

Esta es la gráfica que responde la pregunta que más le importa al jurado: **no “¿cuándo va a quebrar?” sino “¿qué tan seguro está el modelo de eso?”**. El área bajo la curva a la izquierda de la línea naranja es exactamente `prob_quiebra_antes_finiquito` — 46% en este ejemplo.

3.3 Tabla de métricas

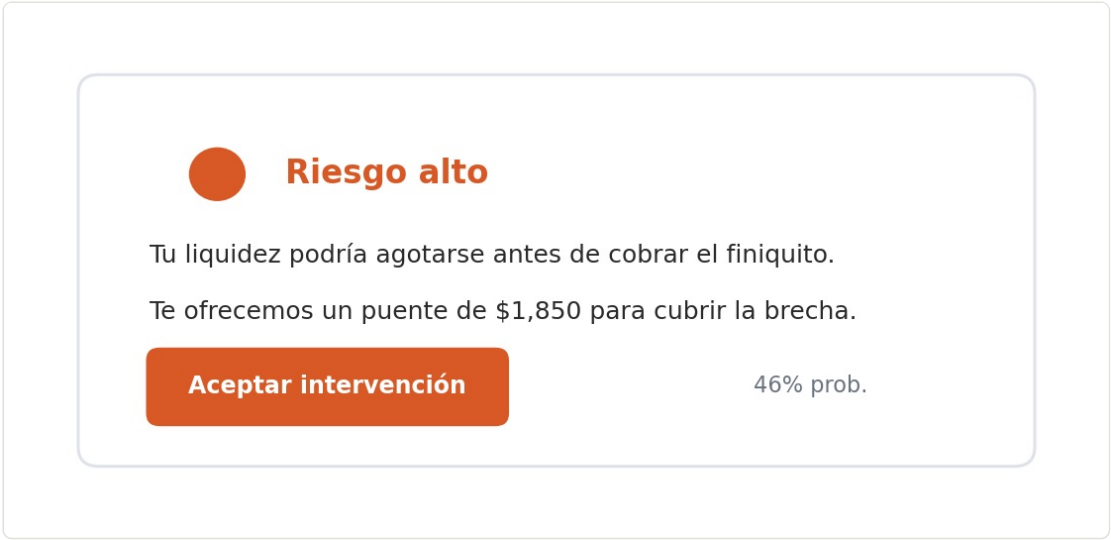
La forma más directa de leer los números exactos del JSON, sin interpretación visual de por medio. Útil

para depuración, para el jurado técnico, o como un “modo avanzado” del dashboard.

Campo del JSON	Etiqueta en la tabla	Valor
parametros.B0	Capital inicial	\$5,000.00
parametros.mu	Drift diario	-\$266.52
parametros.sigma	Volatilidad diaria	\$237.30
prediccion.dia_esperado_quiebra	Día esperado de quiebra	18.76
prediccion.intervalo_confianza_90	Intervalo 90% de confianza	[13.15, 25.69] días
riesgo.fecha_finiquito	Fecha del finiquito	Día 18
riesgo.prob_quiebra_antes_finiquito	Probabilidad de quiebra antes	46.0%
riesgo.estado	Estado	Riesgo alto

3.4 Tarjeta de estado (semáforo)

La vista que realmente le importa al usuario final — el dueño de la PyME no quiere ver una curva de densidad de probabilidad, quiere saber en una frase si está en problemas y qué puede hacer. Esta tarjeta solo lee dos campos del JSON: `riesgo.estado` (para decidir el color del punto) y `riesgo.prob_quiebra_antes_finiquito` (para el texto de apoyo).



Mockup de la tarjeta de estado

El color del punto se decide con una regla simple en el frontend:

```
function colorEstado(estado) {
  const mapa = {
    "seguro": "#0f8f6e", // verde/teal
    "riesgo_medio": "#c98500", // amber
    "riesgo_alto": "#d95926", // coral
  };
  return mapa[estado] || "#6b7280";
}
```

4. Resumen: quién ve qué

Vista	Consume del JSON	Audiencia
Gráfica de trayectoria	<code>serie_saldo</code> , <code>parametros</code>	Dashboard principal, todos los usuarios
Gráfica de distribución	<code>prediccion</code> , <code>riesgo</code>	Jurado técnico, panel de detalle
Tabla de métricas	Todo el JSON, sin transformar	Desarrolladores, modo avanzado
Tarjeta de estado	<code>riesgo.estado</code> , <code>riesgo.prob_quiebra_antes_finiquito</code>	Dueño de la PyME (usuario final)

La idea central para el equipo: la complejidad matemática vive una sola vez, en el backend, dentro del JSON. Cada vista del frontend es solo una manera distinta de dibujar los mismos números — ninguna vuelve a calcular nada. Esto significa que agregar una quinta vista en el futuro (por ejemplo, una notificación push) es tan simple como leer dos o tres campos del mismo contrato, sin tocar el motor matemático ni el pipeline de datos.